

Online Retail Sales Data Engineering

Data Engineering Assessment — Design Report

A reproducible ETL pipeline over 541,909 online-retail transaction rows, with source versioning, validated staging, a DuckDB star schema, data-quality monitoring, and partitioned Parquet analytical storage.

1. Architecture

online_retail.csv	→	Validate + archive	→	raw.online_retail
Raw records	→	Type + standardize	→	Clean staging
Valid sales	→	Star-schema load	→	Dimensions + fact
fact_sales	→	Monitor + partition	→	DuckDB + Parquet

Pipeline lifecycle

Each run validates the expected eight-column source schema, calculates a SHA-256 fingerprint, and archives the exact input. DuckDB ingests source values as text so malformed fields cannot silently change or stop ingestion. Staging applies safe type conversion, standardization, transaction classification, and derived line amounts. Invalid and duplicate records remain available in quarantine tables.

2. Data cleaning and validation

Invoice and stock identifiers are trimmed and standardized; quantities, timestamps, prices, and customer IDs are safely converted to analytical types. Records with missing business identifiers, invalid dates or numbers, zero quantities, or negative prices are quarantined with rejection reasons. Exact duplicates are separated while their first occurrence is retained.

3. Business rules

Invoices beginning with C and negative quantities are classified as returns, preserving their signed financial impact. Missing customer IDs map to an Unknown dimension member so anonymous but valid revenue is not lost. Quantity and price values above the global 99th percentile are flagged for review rather than deleted without business evidence.

4. Data warehouse

The warehouse uses a star schema with product, customer, invoice, and date dimensions surrounding a line-level sales fact table. Surrogate keys create consistent relationships; quantity, unit price, line amount, transaction type, and outlier flags support reporting. A daily aggregate cache accelerates frequently requested time-series summaries.

5. Automation and quality

The same idempotent command runs manually, through Windows Task Scheduler or cron, or in the optional Docker image. Critical checks stop the job when data is unavailable, fact reconciliation fails, or dimension relationships break. Warning thresholds monitor rejection and duplication rates, with results stored for every ETL run.

Verified pipeline results

Raw rows	Clean fact rows	Duplicates	Rejected rows
541,909	536,639	5,268	2
Products	Customers	Invoices	Calendar dates
3,958	4,373	25,898	305

The dataset covers 1 December 2010 through 9 December 2011. The warehouse contains 526,052 sale lines and 10,587 return lines. Five automated integration tests pass, all critical quality checks pass, and clean staging rows reconcile exactly to the fact table.

Versioning, metadata, and lineage

Every distinct input is archived under its SHA-256 version with a manifest. ETL run metadata records status, timestamps, source location, row counts, and errors. Lineage entries document movement from the raw layer through validated staging and the warehouse to versioned Parquet exports, making results reproducible and auditable.

Performance optimization

The fact table is exported as ZSTD-compressed Parquet partitioned by source version, year, and month, matching common time-based reporting filters without producing excessive small files. Fact foreign keys and dimension business keys are indexed. A representative EXPLAIN command exposes the physical join, filtering, and aggregation plan for review.

Limitations

The supplied file is a snapshot without a dependable update watermark, so the current load uses a deterministic full refresh. Product descriptions and customer countries are inferred from transaction history because no mastered reference data was supplied. Global percentile flags are useful screening signals but do not replace product-specific business thresholds.

Production improvements

For larger or concurrent workloads, move the same model to object storage and a managed warehouse or lakehouse. Add incremental ingestion with agreed business keys and watermarks, mastered product and customer dimensions, orchestration with retries and backfills, role-based access, encrypted secrets, centralized observability, and upstream/downstream reconciliation.

Monitoring

Track freshness, runtime, volume, schema drift, missing values, duplicates, rejection rate, referential integrity, query latency, partition size, and dashboard reconciliation. Route critical failures to the operational alerting platform and review warning trends before they become reporting incidents.